

Conditional Probability and Joint Distributions

Table of Contents

1. [Introduction](#)
 2. [Joint Probability](#)
 3. [Marginal Probability](#)
 4. [Conditional Probability](#)
 5. [Relationship Between Joint, Marginal, and Conditional](#)
 6. [Practical Examples](#)
 7. [Computing Conditional PMFs in NumPy](#)
-

Introduction

Often, knowing one piece of information changes the probability of another event. This is called **conditional probability**.

Real-world example:

- Without information: "The submarine could be anywhere in the 16×16 grid"
 - With information: "The submarine's y-coordinate is 4"
 - Now we only need to search 16 squares instead of 256!
-

Joint Probability

Definition

Joint probability is the probability that two (or more) events happen together.

Notation: $P(X = x \text{ AND } Y = y) = P(X = x, Y = y)$

Example: 2D Grid

For a submarine on a 2D grid, the PMF gives us the joint probability:

$$\text{submarine_pmf}[x, y] = P(X_x = x \text{ AND } X_y = y)$$

This tells us: "What's the probability the submarine is at exactly this position $[x, y]$?"

Visualization

Think of a joint PMF as a heatmap where:

- **Bright spots** = High probability the submarine is there
- **Dark spots** = Low probability

	y=0	y=1	y=2	y=3
x=0	0.01	0.02	0.01	0.00
x=1	0.02	0.05	0.03	0.01
x=2	0.01	0.03	0.04	0.02
x=3	0.00	0.01	0.02	0.01

Marginal Probability

Definition

Marginal probability is the probability of one variable, regardless of the other variable(s).

Formula:

$$P(X = x) = \sum_y P(X = x, Y = y) \quad [\text{sum over all } y]$$

$$P(Y = y) = \sum_x P(X = x, Y = y) \quad [\text{sum over all } x]$$

In words: To get the marginal probability, sum out the other variable.

Why "Marginal"?

The name comes from writing probabilities in a table and summing rows/columns, writing totals in the margins:

	y=0	y=1	y=2		Marginal P(X)
x=0	0.1	0.2	0.1		0.4 ← Sum across row
x=1	0.0	0.3	0.3		0.6 ← Sum across row
	-----	-----	-----		
Marginal P(Y)	0.1	0.5	0.4		1.0
	↑	↑	↑		
	Sum down each column				

Example: Submarine X Coordinate

Question: What's the probability the submarine has x-coordinate = 5, regardless of y?

Answer: Sum all probabilities in column x=5

```
p_x_5 = np.sum(submarine_pmf[5, :])
```

This is the marginal probability $P(X_x = 5)$.

Computing All Marginal Probabilities

```
# Marginal PMF of X (sum over y-axis, axis=1)
p_x = np.sum(submarine_pmf, axis=1) # Shape: (16,)

# Marginal PMF of Y (sum over x-axis, axis=0)
p_y = np.sum(submarine_pmf, axis=0) # Shape: (16,)
```

Verify: Both should sum to 1.0

```
assert np.isclose(np.sum(p_x), 1.0)
assert np.isclose(np.sum(p_y), 1.0)
```

Conditional Probability

Definition

Conditional probability is the probability of one event given that another event has occurred.

Notation: $P(A | B)$ reads as "probability of A given B"

Formula:

$$P(A | B) = P(A \text{ AND } B) / P(B) = P(A, B) / P(B)$$

Intuition:

- $P(B)$ is the "universe" of possibilities
- $P(A, B)$ is the overlap
- We're asking: "Within the world where B happened, how much of it is also A?"

Visual Understanding

Imagine a Venn diagram:

Total area = 1.0

Area of B = 0.3

Area of (A AND B) = 0.1

$$P(A \mid B) = 0.1 / 0.3 = 1/3$$

"If we know we're in B, there's a 1/3 chance we're also in A"

Example: Submarine Search

Scenario: Satellite imaging tells us the submarine's y-coordinate is 4.

Question: Given y=4, what's the probability x=3?

$$\begin{aligned} P(X_x = 3 \mid X_y = 4) &= P(X_x = 3, X_y = 4) / P(X_y = 4) \\ &= \text{submarine_pmf}[3, 4] / \text{np.sum(submarine_pmf[:, 4])} \end{aligned}$$

Interpretation: Of all the probability mass on row y=4, what fraction is at position x=3?

Conditional PMFs

Definition

A **conditional PMF** gives the probability distribution of one variable given the value of another.

Notation:

- $P(X \mid Y = y)$: Distribution of X given Y equals a specific value y
- $P(X \mid Y)$: All conditional distributions of X for each value of Y

Example: $P(X | Y = 4)$

Question: If we know $y=4$, what's the distribution over x ?

```
# Joint probabilities where y=4
joint_y4 = submarine_pmf[:, 4] # Shape: (16,)

# Marginal probability y=4
p_y4 = np.sum(joint_y4)

# Conditional PMF  $P(X | Y=4)$ 
p_x_given_y4 = joint_y4 / p_y4 # Shape: (16,)
```

Verify: This should sum to 1.0 (it's a probability distribution)

```
assert np.isclose(np.sum(p_x_given_y4), 1.0)
```

Matrix of All Conditional PMFs: $P(X | Y)$

Goal: For each value of y , compute the distribution over x .

Result: A 16×16 matrix where:

- Each column represents $P(X | Y = y)$ for a specific y
- Column y tells us: "If we know $Y=y$, what's the distribution of X ?"

```
# Method 1: Using broadcasting
p_y = np.sum(submarine_pmf, axis=0) # Marginal P(Y), shape (16,)
p_x_given_y = submarine_pmf / p_y[None, :] # Broadcasting, shape (16, 16)

# Method 2: Explicit loop
p_x_given_y = np.zeros_like(submarine_pmf)
for y in range(16):
    p_y_val = np.sum(submarine_pmf[:, y])
    if p_y_val > 0: # Avoid division by zero
        p_x_given_y[:, y] = submarine_pmf[:, y] / p_y_val
```

Verify: Each column should sum to 1.0

```
column_sums = np.sum(p_x_given_y, axis=0)
assert np.allclose(column_sums, 1.0)
```

Matrix of All Conditional PMFs: $P(Y | X)$

Similarly, for each value of x , compute the distribution over y .

```
# Each row represents  $P(Y | X = x)$ 
p_x = np.sum(submarine_pmf, axis=1) # Marginal  $P(X)$ , shape (16,)
p_y_given_x = submarine_pmf / p_x[:, None] # Broadcasting, shape (16, 16)

# Verify: each row should sum to 1.0
row_sums = np.sum(p_y_given_x, axis=1)
assert np.allclose(row_sums, 1.0)
```

Relationships Between Joint, Marginal, and Conditional

The Chain Rule

$$P(A, B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

In words:

- Joint = Conditional $\times$ Marginal
- "Probability of both = Probability of one $\times$ Probability of other given the first"

Example

```
# These should be equal:
joint = submarine_pmf[3, 4]

# Method 1: P(X=3, Y=4) = P(X=3 | Y=4) · P(Y=4)
p_y4 = np.sum(submarine_pmf[:, 4])
p_x3_given_y4 = submarine_pmf[3, 4] / p_y4
reconstructed_1 = p_x3_given_y4 * p_y4

# Method 2: P(X=3, Y=4) = P(Y=4 | X=3) · P(X=3)
p_x3 = np.sum(submarine_pmf[3, :])
p_y4_given_x3 = submarine_pmf[3, 4] / p_x3
reconstructed_2 = p_y4_given_x3 * p_x3

assert np.isclose(joint, reconstructed_1)
assert np.isclose(joint, reconstructed_2)
```

Sum Rule (Marginalization)

$$\begin{aligned} P(A) &= \sum_b P(A, B = b) \\ &= \sum_b P(A \mid B = b) \cdot P(B = b) \end{aligned}$$

In words: To get the marginal, sum the joint over all values of the other variable.

Conditional Probability with Multiple Conditions

Example: P(Y | X is even)

Question: If we know x is even ($x \in \{0, 2, 4, 6, 8, 10, 12, 14\}$), what's the distribution of y?

Step 1: Find all positions where x is even


```
even_x = np.array([0, 2, 4, 6, 8, 10, 12, 14])
```

Step 2: Get joint probabilities for these x values

```
joint_even_x = submarine_pmf[even_x, :] # Shape: (8, 16)
```

Step 3: Compute marginal $P(X \text{ is even})$

```
p_x_even = np.sum(joint_even_x)
```

Step 4: Marginalize over x to get $P(Y | X \text{ even})$

```
# For each y, sum over all even x values
joint_even_x_each_y = np.sum(joint_even_x, axis=0) # Shape: (16,)

# Normalize
p_y_given_x_even = joint_even_x_each_y / p_x_even # Shape: (16,)
```

Verify: Should sum to 1.0

```
assert np.isclose(np.sum(p_y_given_x_even), 1.0)
```

Example: $P(Y \text{ even} | X \text{ odd})$

Question: If x is odd AND y is even, what's the distribution?

Step 1: Create masks

```
xx, yy = np.meshgrid(np.arange(16), np.arange(16))
mask_x_odd = (xx % 2 == 1)
mask_y_even = (yy % 2 == 0)
combined_mask = mask_x_odd & mask_y_even
```

Step 2: Get joint probabilities

```
joint_x_odd_y_even = submarine_pmf[combined_mask] # 1D array
p_x_odd_y_even = np.sum(joint_x_odd_y_even)
```

But we want the DISTRIBUTION over even y values given x is odd:

```
# Step 1: Get probabilities where x is odd
joint_x_odd = submarine_pmf[:, :].copy()
joint_x_odd[xx % 2 == 0] = 0 # Zero out even x positions

# Step 2: Marginal P(X odd)
p_x_odd = np.sum(joint_x_odd)

# Step 3: For each even y value, sum over odd x values
even_y_indices = np.arange(0, 16, 2) # [0, 2, 4, 6, 8, 10, 12, 14]
p_y_even_given_x_odd = np.zeros(8)

for i, y in enumerate(even_y_indices):
    # Sum probabilities where x is odd and y equals this value
    p_y_even_given_x_odd[i] = np.sum(joint_x_odd[:, y])

# Step 4: Normalize
p_y_even_given_x_odd = p_y_even_given_x_odd / p_x_odd
```

Verify: Should sum to 1.0

```
assert np.isclose(np.sum(p_y_even_given_x_odd), 1.0)
```

Practical Examples

Example 1: Most Likely X Given Y=7

Question: If satellite tells us $y=7$, which x value is most likely?

```
# Conditional distribution P(X | Y=7)
p_x_given_y7 = submarine_pmf[:, 7] / np.sum(submarine_pmf[:, 7])

# Find maximum
most_likely_x = np.argmax(p_x_given_y7)
probability = p_x_given_y7[most_likely_x]

print(f"Given y=7, x={most_likely_x} is most likely with probability {probability}")
```

Example 2: Reducing Uncertainty

Question: Does knowing Y reduce uncertainty about X ?

```
# Uncertainty in X without knowing Y (entropy of marginal)
p_x_marginal = np.sum(submarine_pmf, axis=1)
# (We'll compute entropy in the next guide)

# Average uncertainty in X knowing Y
# For each y, compute entropy of P(X | Y=y), then average
```

We'll cover this in the entropy guide!

Example 3: Independence Test

Two variables are **independent** if:

$$P(X | Y) = P(X) \quad \text{for all } Y$$

Equivalently:

$$P(X, Y) = P(X) \cdot P(Y) \quad \text{for all } X, Y$$

Test:

```
# Compute marginals
p_x = np.sum(submarine_pmf, axis=1)
p_y = np.sum(submarine_pmf, axis=0)

# If independent, joint should equal product of marginals
expected_if_independent = p_x[:, None] * p_y[None, :]

# Compare
difference = np.abs(submarine_pmf - expected_if_independent)
max_difference = np.max(difference)

if max_difference < 1e-10:
    print("X and Y are independent!")
else:
    print(f"X and Y are NOT independent (max difference: {max_difference})")
```

Computing Conditional PMFs in NumPy

Pattern 1: $P(X | Y = \text{specific value})$

```
def conditional_pmf_x_given_y_value(joint_pmf, y_value):  
    """  
    Compute  $P(X | Y = y\_value)$   
  
    Parameters:  
    -----  
    joint_pmf : array (n_x, n_y)  
        Joint probability distribution  
    y_value : int  
        Specific value of Y  
  
    Returns:  
    -----  
    p_x_given_y : array (n_x,)  
        Conditional distribution of X given  $Y=y\_value$   
    """  
    # Get column corresponding to  $Y=y\_value$   
    joint_at_y = joint_pmf[:, y_value]  
  
    # Normalize  
    marginal_y = np.sum(joint_at_y)  
  
    if marginal_y > 0:  
        return joint_at_y / marginal_y  
    else:  
        # If  $P(Y=y\_value) = 0$ , undefined; return uniform or zeros  
        return np.zeros_like(joint_at_y)
```

Pattern 2: $P(X | Y)$ for all Y

```
def conditional_pmf_x_given_y_all(joint_pmf):
    """
    Compute  $P(X | Y)$  for all values of  $Y$ 

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
        Joint probability distribution

    Returns:
    -----
    p_x_given_y : array (n_x, n_y)
        Matrix where column  $y$  is  $P(X | Y=y)$ 
    """
    # Marginal  $P(Y)$ 
    p_y = np.sum(joint_pmf, axis=0) # Shape: (n_y,)

    # Avoid division by zero
    p_y_safe = np.where(p_y > 0, p_y, 1.0)

    # Conditional PMF (broadcast division)
    p_x_given_y = joint_pmf / p_y_safe[None, :]

    # Zero out columns where  $P(Y=y)$  was 0
    p_x_given_y[:, p_y == 0] = 0

    return p_x_given_y
```

Pattern 3: $P(Y | X)$ for all X

```
def conditional_pmf_y_given_x_all(joint_pmf):
    """
    Compute  $P(Y | X)$  for all values of  $X$ 

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
        Joint probability distribution

    Returns:
    -----
    p_y_given_x : array (n_x, n_y)
        Matrix where row  $x$  is  $P(Y | X=x)$ 
    """
    # Marginal  $P(X)$ 
    p_x = np.sum(joint_pmf, axis=1) # Shape: (n_x,)

    # Avoid division by zero
    p_x_safe = np.where(p_x > 0, p_x, 1.0)

    # Conditional PMF (broadcast division)
    p_y_given_x = joint_pmf / p_x_safe[:, None]

    # Zero out rows where  $P(X=x)$  was 0
    p_y_given_x[p_x == 0, :] = 0

    return p_y_given_x
```

Pattern 4: $P(Y \mid X \text{ satisfies condition})$

```
def conditional_pmf_y_given_x_condition(joint_pmf, x_mask):
    """
    Compute  $P(Y \mid X \text{ satisfies some condition})$ 

    Parameters:
    -----
    joint_pmf : array (n_x, n_y)
        Joint probability distribution
    x_mask : boolean array (n_x,) or list of indices
        Which x values satisfy the condition

    Returns:
    -----
    p_y_given_condition : array (n_y,)
        Conditional distribution of Y given condition on X
    """
    # Get joint probabilities where condition holds
    if isinstance(x_mask, (list, np.ndarray)) and x_mask.dtype != bool:
        # List of indices
        joint_condition = joint_pmf[x_mask, :]
    else:
        # Boolean mask
        joint_condition = joint_pmf[x_mask, :]

    # Marginalize over x: sum over rows
    joint_y_and_condition = np.sum(joint_condition, axis=0) # Shape: (n_y,)

    # Marginal P(condition)
    p_condition = np.sum(joint_y_and_condition)

    # Conditional PMF
    if p_condition > 0:
        return joint_y_and_condition / p_condition
    else:
        return np.zeros(joint_pmf.shape[1])
```


Example usage:

```
# P(Y | X is even)
even_x_mask = np.arange(16) % 2 == 0
p_y_given_x_even = conditional_pmf_y_given_x_condition(submarine_pmf, even_x_m
```

Common Mistakes to Avoid

Mistake 1: Forgetting to normalize

```
# WRONG: This is the joint, not conditional!
p_x_given_y4 = submarine_pmf[:, 4]

# CORRECT: Divide by marginal
p_y4 = np.sum(submarine_pmf[:, 4])
p_x_given_y4 = submarine_pmf[:, 4] / p_y4
```

Mistake 2: Confusing $P(X|Y)$ with $P(Y|X)$

These are generally NOT equal!

```
p_x_given_y = submarine_pmf / np.sum(submarine_pmf, axis=0)[None, :]
p_y_given_x = submarine_pmf / np.sum(submarine_pmf, axis=1)[:, None]

# These are different matrices!
assert not np.allclose(p_x_given_y, p_y_given_x)
```

Mistake 3: Wrong axis for summation

```
# For P(X | Y): sum over x-axis (axis=0) to get P(Y)
# NO! Sum over axis=0 gives sum over rows (x values) for each y

# CORRECT for P(X | Y):
p_y = np.sum(joint, axis=0) # Sum each column
p_x_given_y = joint / p_y[None, :]

# CORRECT for P(Y | X):
p_x = np.sum(joint, axis=1) # Sum each row
p_y_given_x = joint / p_x[:, None]
```

Remember:

- axis=0 sums down (across rows)
- axis=1 sums across (across columns)

Mistake 4: Not handling zero probabilities

```
# WRONG: Division by zero error
p_x_given_y = joint / np.sum(joint, axis=0)[None, :]

# CORRECT: Handle zeros
marginal_y = np.sum(joint, axis=0)
marginal_y_safe = np.where(marginal_y > 0, marginal_y, 1.0)
p_x_given_y = joint / marginal_y_safe[None, :]
p_x_given_y[:, marginal_y == 0] = 0 # Set undefined conditionals to 0
```

Verification Checklist

When computing conditional probabilities, always verify:

```
# ✓ Conditional PMF sums to 1
assert np.allclose(np.sum(p_x_given_y, axis=0), 1.0) # Each column sums to 1
assert np.allclose(np.sum(p_y_given_x, axis=1), 1.0) # Each row sums to 1

# ✓ All probabilities in [0, 1]
assert np.all(p_x_given_y >= 0) and np.all(p_x_given_y <= 1)

# ✓ Chain rule holds:  $P(X,Y) = P(X|Y) \cdot P(Y)$ 
p_y = np.sum(joint, axis=0)
reconstructed = p_x_given_y * p_y[None, :]
assert np.allclose(joint, reconstructed)

# ✓ Marginalization works:  $P(X) = \sum_y P(X|Y=y) \cdot P(Y=y)$ 
p_x_marginal = np.sum(joint, axis=1)
p_x_from_conditional = np.sum(p_x_given_y * p_y[None, :], axis=1)
assert np.allclose(p_x_marginal, p_x_from_conditional)
```

Summary

Key Concepts:

1. **Joint $P(X, Y)$** : Probability both events occur
2. **Marginal $P(X)$** : Probability of X alone (sum out Y)
3. **Conditional $P(X|Y)$** : Probability of X given Y occurred
4. **Formula**: $P(X|Y) = P(X, Y) / P(Y)$
5. **Chain Rule**: $P(X, Y) = P(X|Y) \cdot P(Y)$

Key NumPy Patterns:

- Marginal: `np.sum(joint, axis=...)`
- Conditional: `joint / marginal` (with broadcasting)
- Always normalize to ensure probabilities sum to 1
- Handle zero divisions carefully

Practice Tips:

1. Draw diagrams for small examples
2. Verify your results sum to 1
3. Check the chain rule: joint = conditional $\times$ marginal
4. Remember: $P(X|Y) \neq P(Y|X)$ in general
5. Be careful with axis ordering in NumPy

Next Topics:

- Bayes' Rule: How to "flip" conditionals
- Independence and correlation
- Conditional expectation